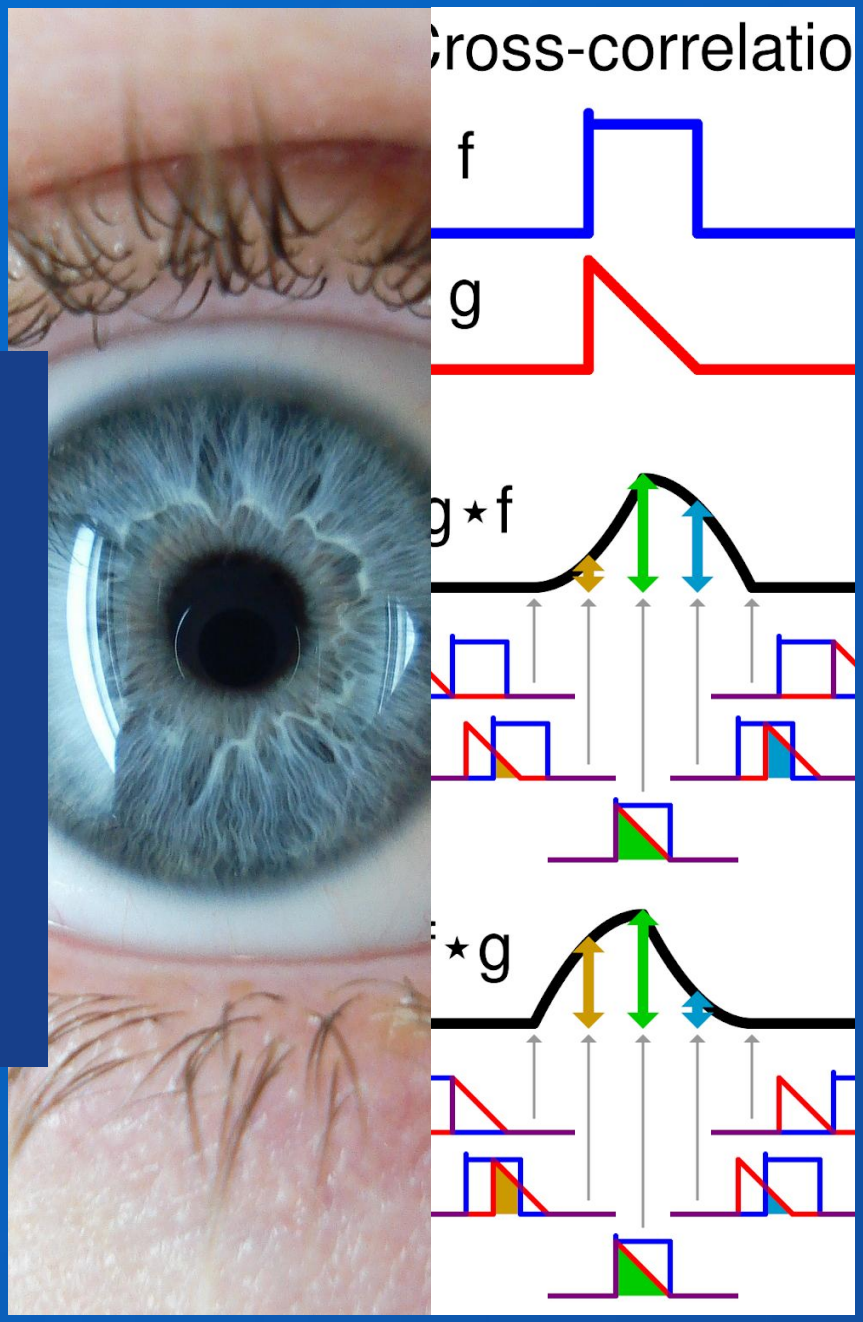
ESCUELA DE INGENIERÍA
FACULTAD DE INGENIERÍA

EDUCACIÓN PROFESIONAL

# Redes Convolucionales

## Expertas en patrones locales

Profesor:
**Alain Raymond**



# Agenda de Hoy

1. ¿Por qué no más MLPs?
2. Redes Convolucionales (CNNs)
3. Otros módulos para CNNs
4. ¿Cómo armamos una red neuronal profunda?

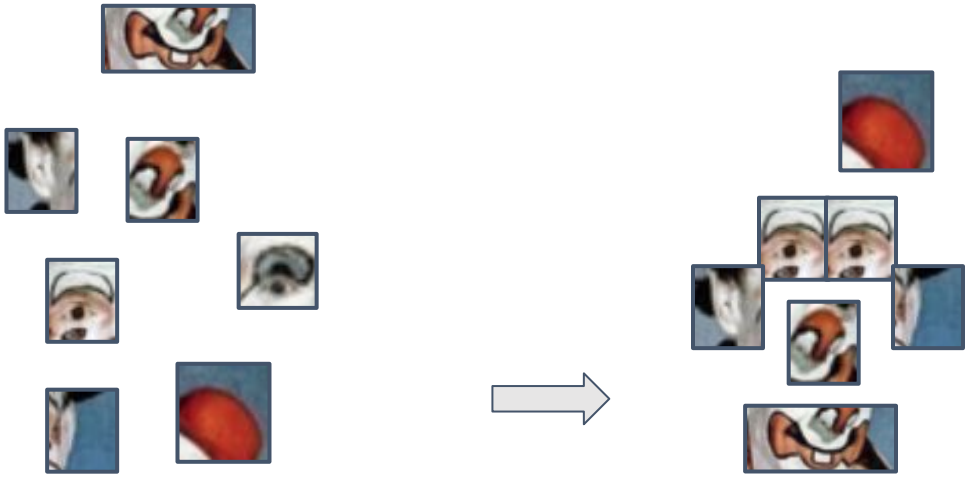
# ¿Por qué no ocupar MLPs más profundos?

- Las distintas capas de una red tienen el trabajo de aprender a **detectar patrones** sobre su entrada.
- El tipo de patrones que pueden aprender las capas lineales no es necesariamente el más adecuado para una tarea particular.
- Se han desarrollado nuevos tipos de capas que pueden aprender **distintos tipos de patrones**.
- Mucho de los avances en el área tienen que ver con haber encontrado la capa correcta para un cierto tipo de problema.

# Patrones Capas Lineales

- Capas lineales aprenden patrones sobre la entrada completa.
- Desde el punto de una capa lineal, cada una de las imágenes de caras a la derecha es un **patrón** distinto.

# Patrones Globales vs Locales



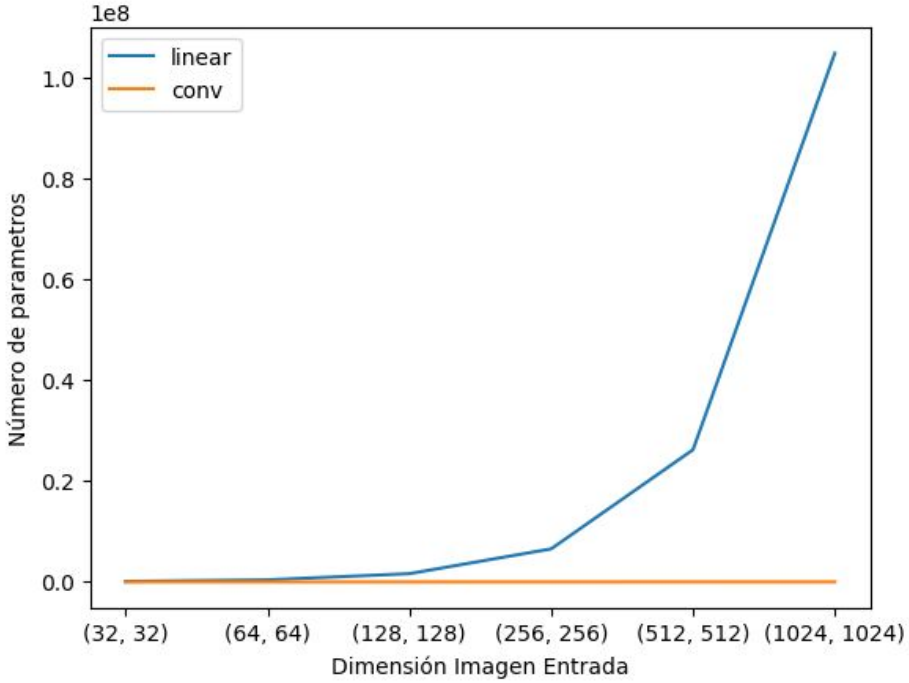
**Globales**

**Locales**

¡Queremos aprender **patrones** que nos permitan **generalizar** a más casos!

# Problema: Capas Lineales son costosas

- Capas Lineales lo que hacen es multiplicar la entrada por una matriz de parámetros y aplicar una función de activación.
- La cantidad de parámetros de esta matriz depende del tamaño de la entrada y de la salida.




n_mlp_params = dim_in x dim_out

**Cantidad de Parámetros para distintas dimensiones de entrada y dimensión salida = 100**

**Capa Lineal vs Capa Convolucional**

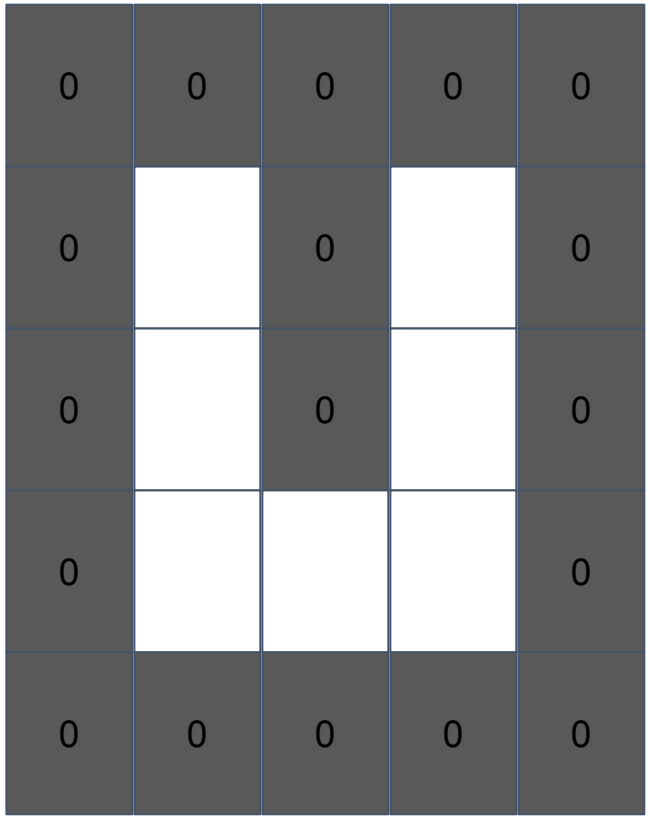
# Solución: Capas convolucionales

- Son una forma alternativa de computar capas ocultas de una red neuronal.
  - Su caso de uso más usual es para trabajar con imágenes.
- Requieren menos parámetros para operar que su equivalente Lineal.
- Son buenas para aprender **patrones locales** en las entradas.
- Una red convolucional es simplemente una red neuronal que contiene capas convolucionales.

# Solución: Capas convolucionales

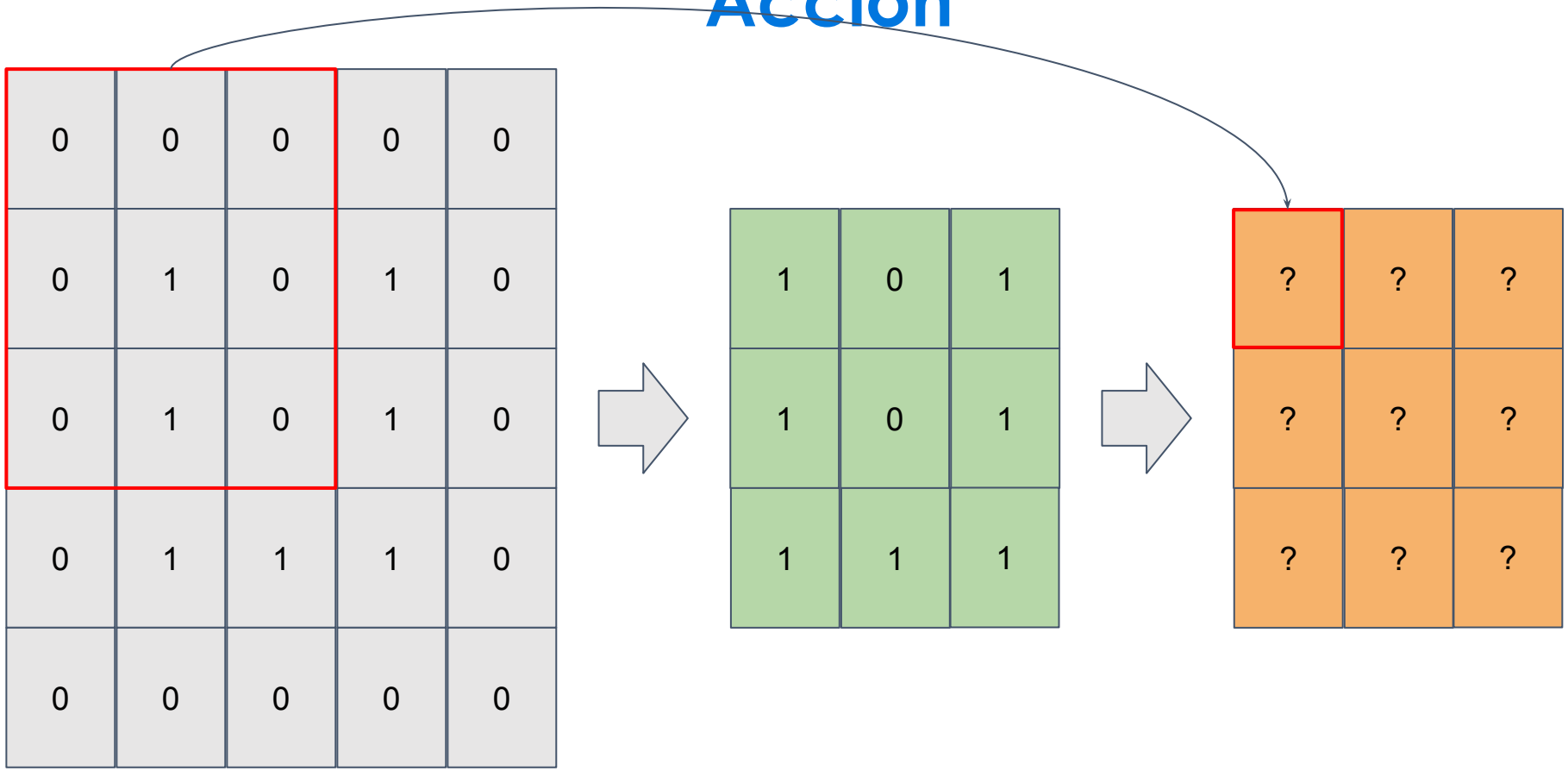
- En vez de agrupar nuestros parámetros en una matriz, los agrupamos en un conjunto de **filtros**.
- Estos **filtros** son típicamente más pequeños que el tamaño de la entrada.
- Luego estos filtros se "aplican" a la entrada en distintas partes de ésta, hasta que se apliquen en toda la entrada. Cada vez que el filtro se aplica esto genera un resultado que llamaremos **neurona**.
- El conjunto de neuronas que da como resultado aplicar el filtro por toda la imagen se llama **mapa de activación**.
- ¡Veamos cómo opera!

# Solución: Capas Convolucionales




| | | | | |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 1 | 0 |
| 0 | 0 | 0 | 0 | 0 |

# Capas Convolucionales en Acción




**Entrada**

**Filtro (Parámetros)**

**Mapa Activación**

# Operación de Convolución en Detalle

| | | |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 0 | 1 | 0 |

**Entrada**

$\times$

| | | |
|---|---|---|
| 1 | 0 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

**Filtro (Parámetros)**

$=$

| | | |
|---|---|---|
| 0*1 | 0*0 | 0*1 |
| 0*1 | 1*0 | 0*1 |
| 0*1 | 1*1 | 0*1 |

**Multiplicación término a término**

$\Rightarrow$

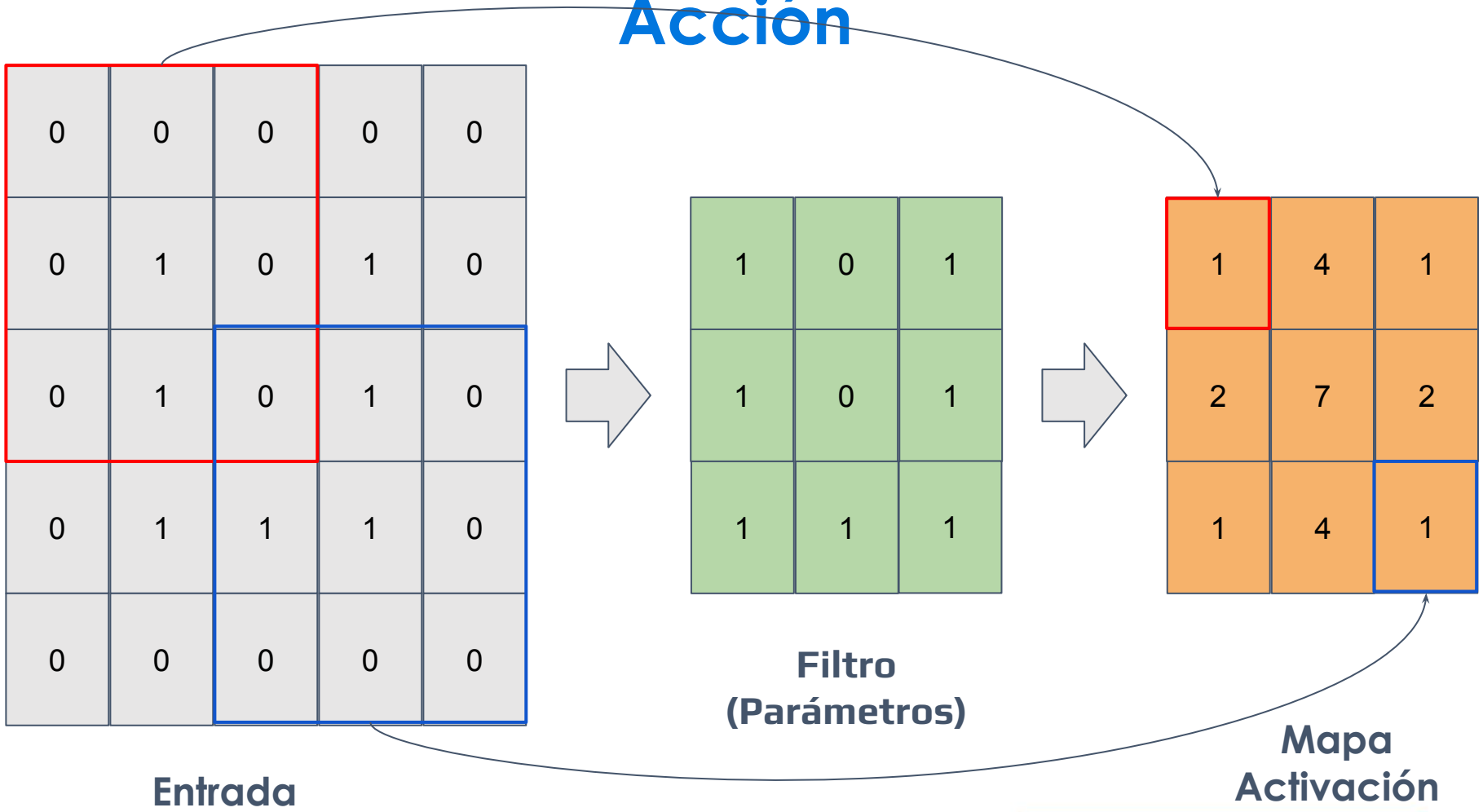
| | | |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 0 | 0 |
| 0 | 1 | 0 |

$\Downarrow$

| |
|---|
| 1 |

**Suma**

# Capas Convolucionales en Acción



# Capas Convolucionales en Acción



¡Filtros son **aprendidos** vía backpropagation ahora!

| | | | | |
|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 0 | 1 | 0 |
| 0 | 1 | 1 | 1 | 0 |
| 0 | 0 | 0 | 0 | 0 |

**Entrada**

| | | |
|---|---|---|
| 1 | 0 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

**Filtro (Parámetros)**

| | | |
|---|---|---|
| 1 | 4 | 1 |
| 2 | 7 | 2 |
| 1 | 4 | 1 |

**Mapa Activación**

# Conceptos Claves de Convolución

| | | |
|---|---|---|
| 1 | 0 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

**Filtro (Parámetros)**

- Tamaño del Filtro se llama **Kernel Size.** En este caso es de **(3x3)**
- Cuanto me muevo antes de aplicar el filtro es un hiperparámetro llamado stride. En este caso, **stride=1**.
- Uno define la cantidad de filtros a aplicar.
- **Importante**: los filtros se aplican sobre todos los canales de entrada. En el caso de imágenes a color, el filtro anterior sobre la entrada sería de tamaño (3 x 3 x 3).
- A veces, para tratar de aprovechar patrones en los bordes de una imagen, esta se rellena con zeros en los bordes. Este concepto se conoce como **zero padding**.

# Cambio de dimensionalidad de la entrada con una capa convolucional

**H** = Ancho o Alto imagen, **K**: Kernel Size, **P**: cantidad de Padding, **S**= Stride

| | | |
|---|---|---|
| 1 | 0 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 1 |

**Filtro (Parámetros)**

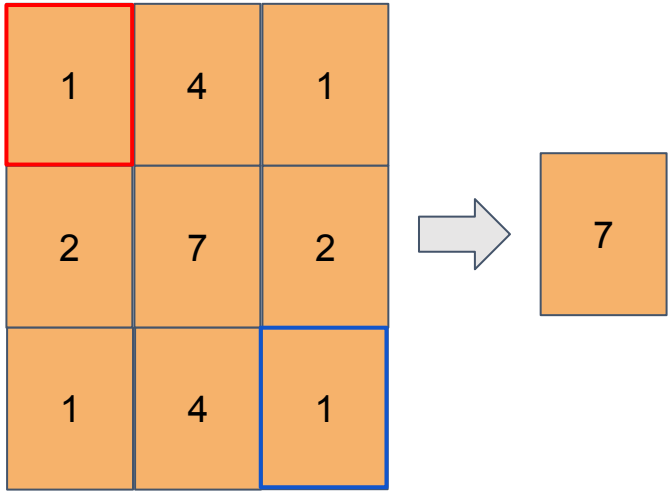
$$O(H) = \left\lfloor \frac{H - K + 2P}{S} \right\rfloor + 1$$

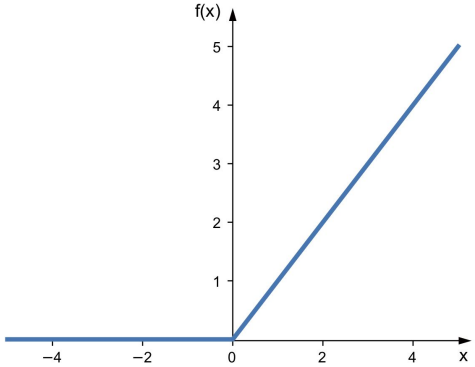
En nuestro caso anterior sería:

$$O(5) = \left\lfloor \frac{5 - 3 + 2 * 0}{1} \right\rfloor + 1 = 3$$

# Dos módulos nuevos que aparecieron al mismo tiempo

- **MaxPooling**: muy similar a capa convolucional, pero simplemente retorna el valor máximo de la entrada.
  - No tiene parámetros.
  - Usada para reducir dimensionalidad de entrada.
  - Hiperparámetros: Kernel Size, Stride, Padding.
- **ReLU**: función de activación más usada en Deep Learning. Consiste en la función ReLU(x) = Max(0, x).
  - Aprende más rápido que Sigmoide.
  - Fácil de computar.



# Redes Convolucionales

¡Son simplemente redes neuronales que contienen **capas convolucionales**!

# ¿Cómo está estructurada una Red Neuronal Profunda?

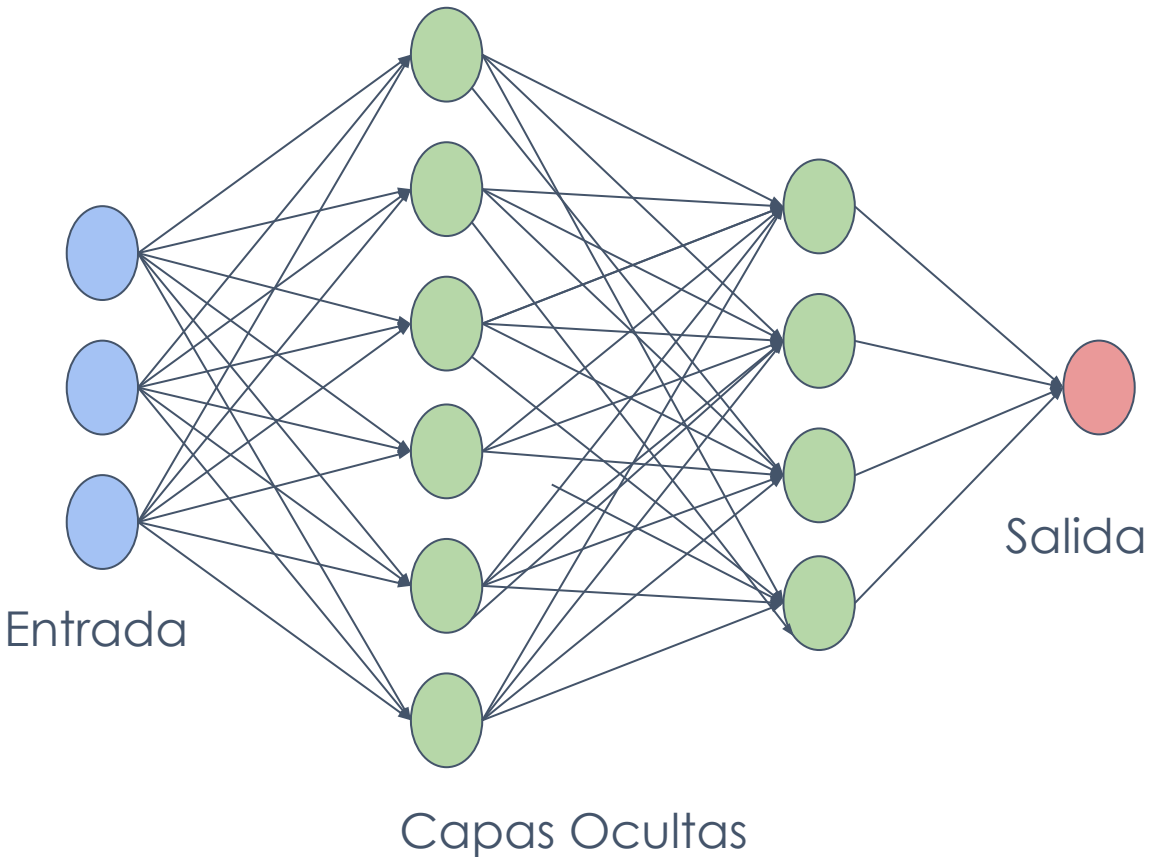
- Una red profunda es una colección de **módulos** que (usualmente) se ejecutan de forma secuencial.
- Cada **módulo** toma como entrada una representación numérica y retorna una **transformación** de esa representación.
  - Esta transformación usualmente altera la dimensionalidad de la representación.

# ¿Cómo está estructurada una Red Neuronal Profunda?

- ¡Ustedes ya conocen un par de módulos!
  - **Lineal**: ¡módulo usado en los MLP! ¡Todos contra todos!
  - **Sigmoide**: función de activación.
- Los módulos se pueden conectar entre ellos usualmente de manera libre.
  - La única restricción es que calcen las dimensionalidades que espera recibir un módulo con la dimensionalidad de la salida del módulo predecesor.

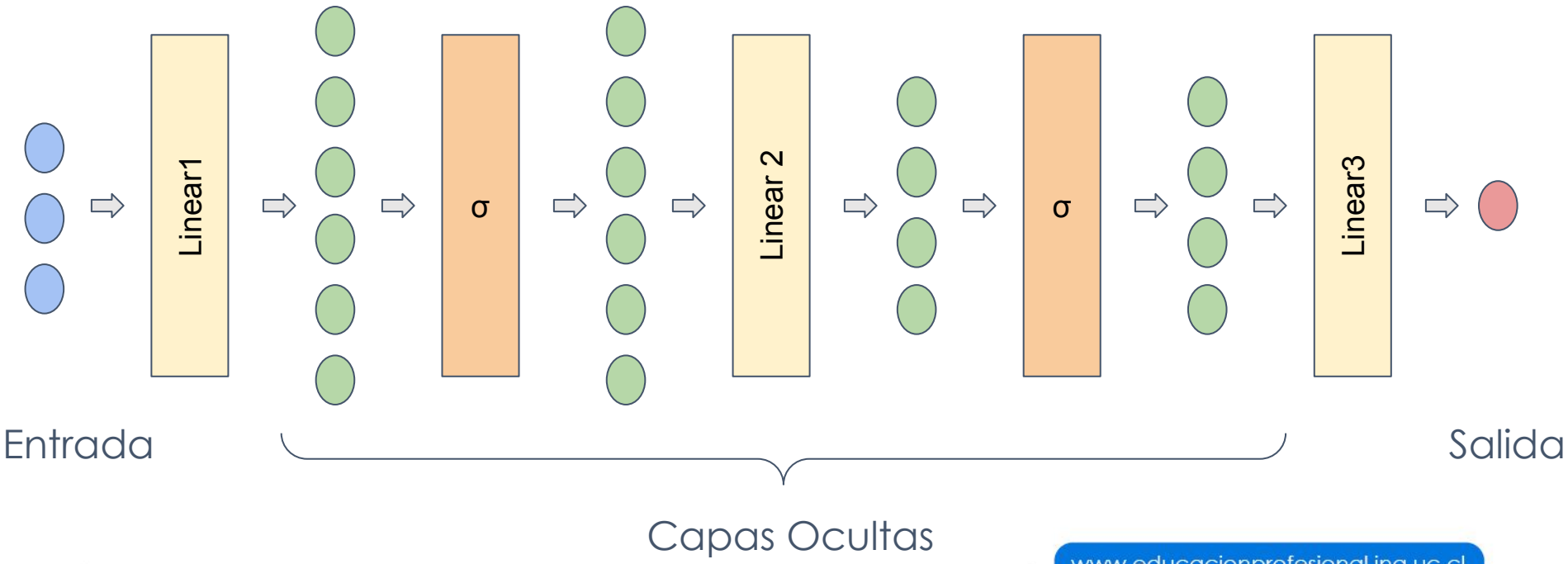
# ¿Cómo está estructurada una Red Neuronal Profunda?

- **Ejemplo:** Multilayer Perceptron (MLP).
- Entrada de tamaño 3, salida tamaño 1.
- Capas ocultas de 6 y 4 neuronas.
- Función de activación Sigmoide.



# ¿Cómo está estructurada una Red Neuronal Profunda?

- Un MLP es una secuencia de módulos Lineales con funciones de activación intercaladas.
- En la práctica esto se ve más o menos así:



# ¿Cómo está estructurada una Red Neuronal Profunda?

- En pseudocódigo se ve así:

```
# x es nuestra entrada y es de tamaño 3
x = Linear1(x)  #x ahora es de tamaño 7
x = Sigmoid(x)  #x ahora es de tamaño 7
x = Linear2(x)  #x ahora es de tamaño 5
x = Sigmoid(x)  #x ahora es de tamaño 5
output = Linear3(x)  # output es de tamaño 1
```

# Ejemplos de Módulos



| Módulo | Entrada | Salida | Comentario |
|---|---|---|---|
| **Lineal** | Definida por usuario | Definida por usuario | Usualmente usada en la parte final de una red. |
| **Sigmoide** | No le importa | Misma que entrada | Función Activación tradicional |
| **MaxPooling2D** | (C_In, H, W) | (C_In, O(H), O(W)) | Capa típicamente usada para manejar audio crudo. |
| **Convolucional2D** | (C_Out, H, W) | (C_out, O(H), O(W)) | Capa típicamente usada para manejar imágenes o espectrogramas de audio. |
| **ReLU** | No le importa | Misma que entrada | Función Activación más usada en la práctica. |

ESCUELA DE INGENIERÍA
FACULTAD DE INGENIERÍA

EDUCACIÓN PROFESIONAL

www.educacionprofesional.ing.uc.cl